



Technical Analysis

Ready Player Pi

Attachment 2.1

Aarhus University

Department of Electrical & Computer Engineering

Bachelor of Engineering in Software technology



AARHUS UNIVERSITET

TABLE OF CONTENTS

Technical Analysis Ready Player Pi.....	1
Objective	3
1. Technical analysis	3
1.1 Controller/RPi	3
1.2 Display	4
1.3 Microphone.....	4
1.4 Accelerometer / Gyroscope	5
1.5 Power Supply	5
1.6 Protocols	6
1.7 Programming languages and libraries	7

Objective

JAPAPDAPWPDW

BILLEDE AF OPSTILLINGEN MED PILE TIL DIVERSE KOMPONENTER

1. Technical analysis

1.1 Controller/RPi

Flagship series:

Runs a full linux operating system and a variety of ports. Usually used for general computing. Has between 2-8 GB memory and quad-core processor.

Compute model:

Hardware equivalent to corresponding flagship models, but with fewer ports and no GPIO pins.

Zero:

Small board with low power consumption, good for small projects. Usually used for embedded projects where size and price matter. Has 512 MB RAM and a single core processor.

We have chosen to use the flagship RPi's they offer multiple core processors and have ports and pins we need for our microphones. The compute model could be used for the server RPi for a cheaper option. Since we already had the flagship RPi's available the compute model wasn't cheaper for the prototype. If later the system should be mass produced the compute model would be a better choice for the server.

<https://www.raspberrypi.com/news/raspberry-pi-product-series-explained/>

<https://www.wonderfulpcb.com/blog/raspberry-pi-models-comparison-features-specs-performance/>

1.2 Display

The system needs a display on which the GUI from the GameApp is displayed.

Laptop:

A laptop could be used to run the GameApp. An advantage is that most people have a laptop available, therefore the system didn't need it's on display. Making it a cheaper option.

Touchscreen:

A small touchscreen makes it possible to make an embedded Gamecontroller containing all that is needed pr. Player. The touchscreen results in a bigger power requirement since they doesn't have their own battery.

We believe that the embedded Gamecontroller results in a better user experience and have therefore chosen to use those. Specifically the Raspberry Pi 7inch touchscreen Display.

1.3 Microphone

MEMS Microphone:

Compact and low power consumption makes it popular for embedded applications. They are often more durable and resistant to shock, moisture and dust. They offer high sensitivity and sound quality.

Electret Microphones

They offer higher sensitivity and audio quality. They are larger and require more power than MEMS microphones. The better quality makes them popular for professional recording and broadcasting.

We have chosen to use a MEMS microphone, since it offers good enough sound quality for our purpose. They are smaller and easy to embed into our Gamecontroller. Specifically we have chosen the Adafruit I2S MEMS Microphone - SPH0645LM4H.

<https://sistc.com/mems-microphone-vs-electret-microphone-comparison/>

1.4 Accelerometer / Gyroscope

The system utilizes the ECE-hat provided in the course *Systemprogramming*, which serves as the hardware foundation for our embedded Game Controller. This hat is equipped with the **BMI160**, a versatile 6-axis Inertial Measurement Unit (IMU) featuring both a 3D gravity accelerometer and a 3D gyroscope.

Usage in GyroPi

To achieve precise and responsive tilt controls for the "GyroPi" game, the system actively combines the data from both the accelerometer and the gyroscope using a sensor fusion algorithm. Relying solely on the accelerometer would result in less accurate data sensitive to the player's hand movements, while using only the gyroscope would introduce angular drift over time. By fusing the two measurements, the Performer Player's RPi calculates a highly stable, real-time tilt angle. This ensures smooth and accurate directional movement of the ball on the Communicator Player's screen, which is critical for the gameplay experience.

The choice to base our Game Controller around the BMI160 is grounded in two main technical advantages:

- **Low Power and Compact Form Factor:** The BMI160 is specifically designed for low-power operation and has a minimal physical footprint, making it highly suitable for a handheld, embedded device.
- **High-Speed Communication:** The module supports SPI (Serial Peripheral Interface) communication. SPI allows for high-frequency, low-latency data polling, which is essential for maintaining a responsive game loop and sending frequent JSON updates over TCP sockets without bottlenecking the system.

https://www.adeept.com/bmi160_p0137.html

1.5 Power Supply

To transform the Raspberry Pi into a standalone Game Controller, we power it using a portable power bank. A traditional wired power supply would tether the player to a wall outlet, restricting mobility and lowering the gaming experience. GyroPi specifically requires this feature to an extent, however in possible expansions of the system, the usage of power banks enable all sorts of games. Thus, integrating a power bank ensures complete wireless freedom, enabling unhindered movement and a smooth gameplay experience anywhere.

DET KONKRET VALG AF POWERBANK

Overvej UPS Hat

Power banks

Charger

1.6 Protocols

Protocols

SPI eller I2C gyroscope.

The gyroscopes default protocol is I2C (p89 in datasheet)

We use SPI in SYS and have configured so via the raspberry pi.

The microfone we have chosen uses I2S as protocol and this

Communication

The system requires a real-time two-way communication between three different RPi's. That includes our server RPi and the two RPi clients needed to play one of the games. Data that should be transmitted includes tilt data from the gyroscope, classification of note data, game events, and session data. The following communication methods have been discussed:

MQTT (Message Queuing Telemetry Transport)

MQTT is a lightweight publish/subscribe messaging protocol using a centralized message broker where clients publish messages to topics, and other clients subscribe to topics, they want messages from. The advantages of using MQTT in this project would be that we minimize overhead and the protocol furthermore supports features such as persistent sessions, last will messages and retained messages. Some of the disadvantages would be that an instance in the form of a broker will eventually run as a separate service. For this project, it will result in a fourth process or a completely dedicated RPi for just having the broker running. This project includes lobby rooms, room codes, and other session-based communication that will not suit a communication method such as this where the focus is supporting millions of devices at the same time using low bandwidth networks.

TCP (Transmission Control Protocol)

TCP is a network communication protocol located in the transportation layer of the OSI reference model. TCP can leverage data guaranteed without any loss and in the current order due to its three-way handshake technology. The disadvantage of using this protocol over

UDP would be that it would be slightly slower in the gaming performance due to the three-way handshake where a few packet losses do not influence the user experience. However, for the NotePi game we want to make sure the data is leveraged to the Server without any packet loss and in the correct order and therefore the TCP would have a good purpose for this scenario and the same goes for all our other data such as leaderboard data and other important session data that needs to be synchronized accross devices.

UDP (User Datagram Protocol)

UDP is a network communication protocol located in the transportation layer of the OSI reference model. UDP does not guarantee the data is getting delivered in order, neither does it guarantee reliability. The advantage is that it does have a minimal overhead as there is no handshake, no work

Konklusion

Kilde mellem TCP mellem UDP: <https://www.pubnub.com/guides/tcp-vs-udp/>

Transport Layer - General, Aarhus University, School of Engineering (Kapitel 3: Transport-layer services, UDP, Reliable data transfer, TCP).

Application Layer - Socket Programming, Aarhus University, School of Engineering (Kapitel 2.3: Socket Programming med UDP og TCP).

Link til MQTT powerpoint når han frigiver den på brightspace.

MQTT information: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

1.7 Programming languages and libraries

C++

Library for GUI

GTKmm

C++ wrapper to GTK+. Developed for Linux. Slower because of the use of wrapper. Not ideal for using in game development.

QT

One of the most popular GUI libraries for C++ therefore a lot of online documentation and tutorials. Good IDE for development. Faster design and development, for programmers without QT experience. There is some group members with previous QT experience. QT can be used in game development. Big framework ...

One of the cons is that it uses old C++ paradigms.

wxWidgets

Is a C++ library. It can be used to cross platform applications. One of the main pros is that it is heavily used and tested by others. The main con is a complicated api, making it difficult to communicate with the rest of our system.

Dear ImGUI

C++ library with no external dependencies. Easy to use interface hard to change the looks of widgets. Difficult to customize. Since our GUI will contain games, we will need some additional libraries for rendering.

<https://thegabmeister.com/p/game-launcher-qt/>

<https://peerdh.com/blogs/programming-insights/creating-a-qt-based-game-with-simple-mechanics>

[https://philippegroarke.com/posts/2018/c++ ui solutions/](https://philippegroarke.com/posts/2018/c++_ui_solutions/)

Flere trade

Threads

Processes

Sockets

